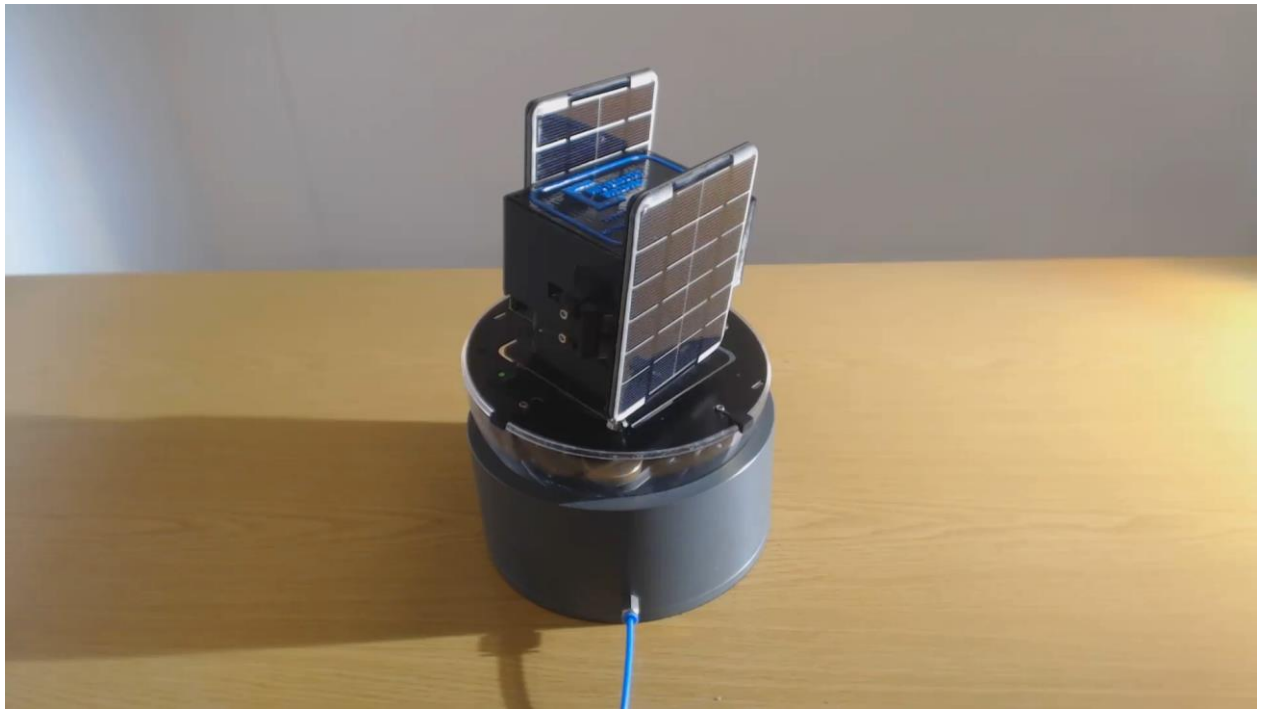
Laboratory Experiments

Laboratory Experiments



Laboratory Experiments

Goal

Laboratory Experiments

1- Toolchain setting + Basic Debugging Techniques

2- STM32F4 Introduction (GPIO and Timers)

3- Telemetry and Telecommand (UART)

4- IMU Interfacing (I2C and SPI)

5- AHRS with Ground Station.

Laboratory Experiments

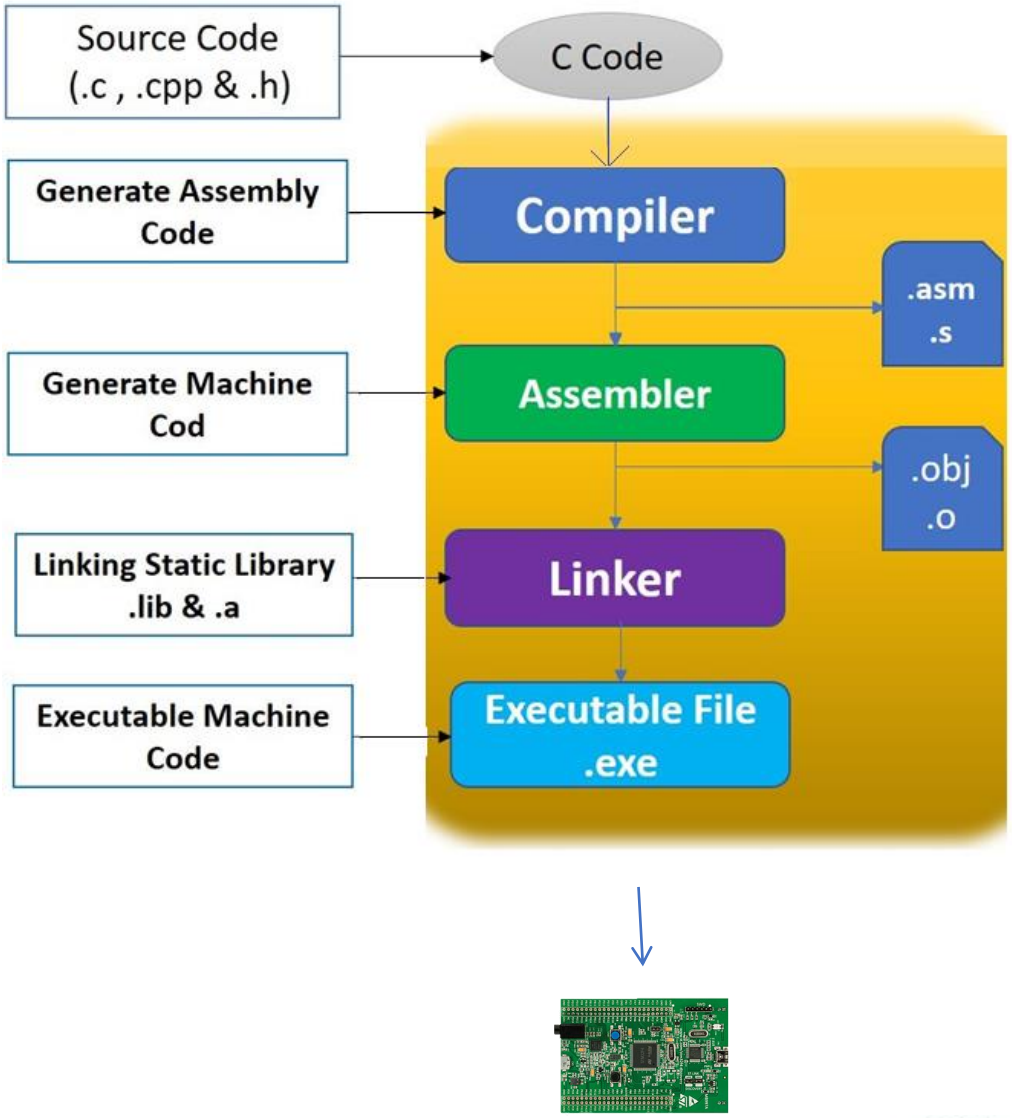
1- Toolchain setting + Basic Debugging Techniques

2- STM32F4 Introduction (GPIO and Timers)

3- Telemetry and Telecommand (UART)

4- IMU Interfacing (I2C and SPI)

5- AHRS with Ground Station.



Laboratory Experiments

1- Toolchain setting + Basic Debugging Techniques

2- STM32F4 Introduction (GPIO and Timers)

3- Telemetry and Telecommand (UART)

4- IMU Interfacing (I2C and SPI)

5- AHRS with Ground Station.

Timer
(HAL_PWM)

GPIO
(HAL_GPIO)

Laboratory Experiments

1- Toolchain setting + Basic Debugging Techniqu

2- STM32F4 Introduction (GPIO and Timers)

3- Telemetry and Telecommand (UART)

4- IMU Interfacing (I2C and SPI)

5- AHRS with Ground Station.

Debugging

UART

(HAL_UART)

Telemetry & Telecommand

Laboratory Experiments

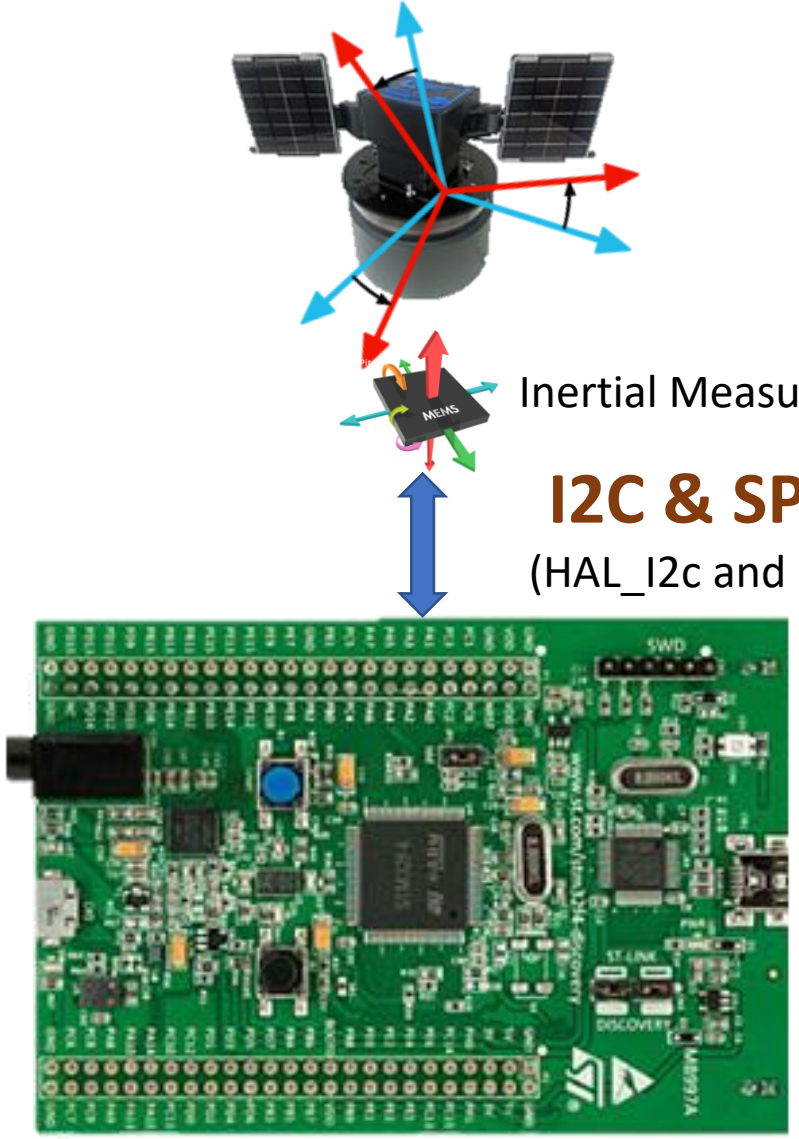
1- Toolchain setting + Basic Debugging Techniques

2- STM32F4 Introduction (GPIO and Timers)

3- Telemetry and Telecommand (UART)

4- IMU Interfacing (I2C and SPI)

5- AHRS with Ground Station.



Laboratory Experiments

1- Toolchain setting + Basic Debugging Techniques

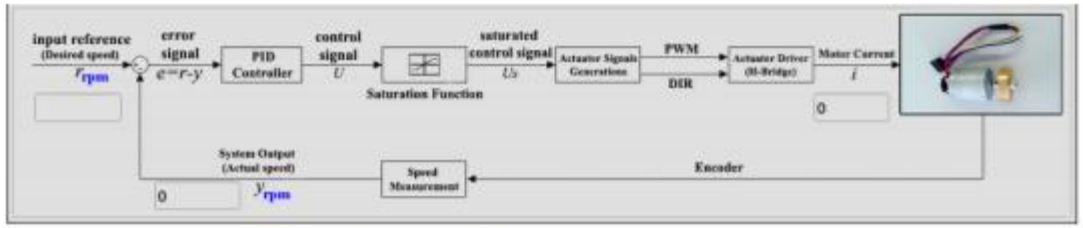
2- STM32F4 Introduction (GPIO and Timers)

3- Telemetry and Telecommand (UART)

4- IMU Interfacing (I2C and SPI)

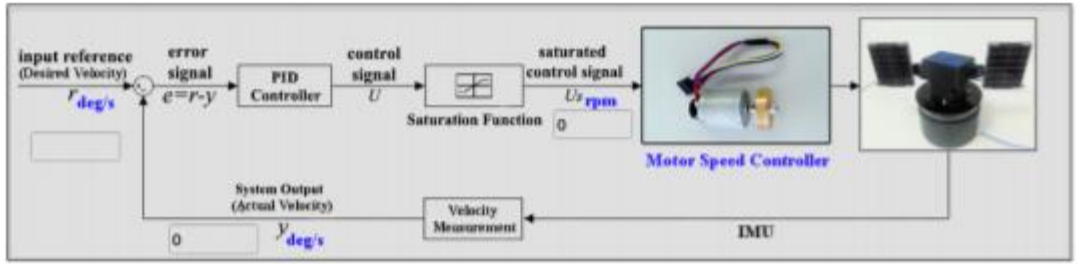
5- AHRS with Ground Station.

Laboratory Experiments

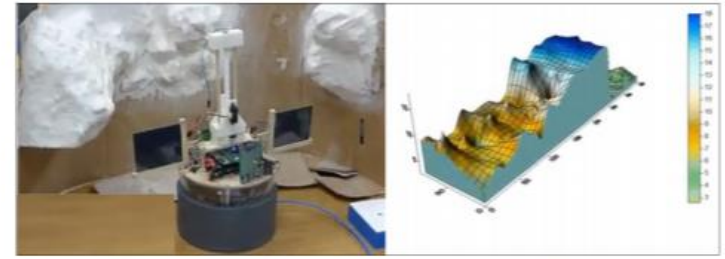


Motor Speed Controller

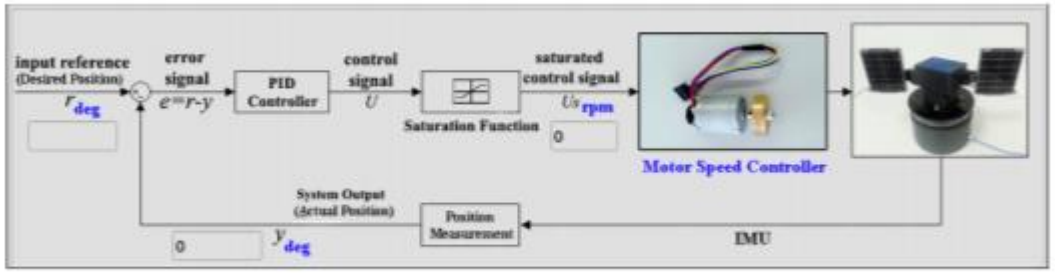
Star Tracking using FloatSat



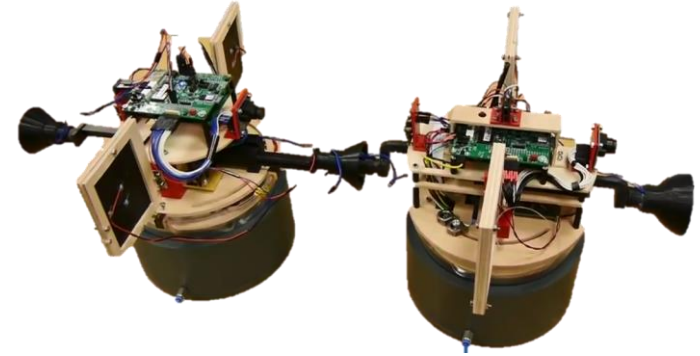
Velocity Controller



Three dimensional scanning of an Analogues surface of a Planet



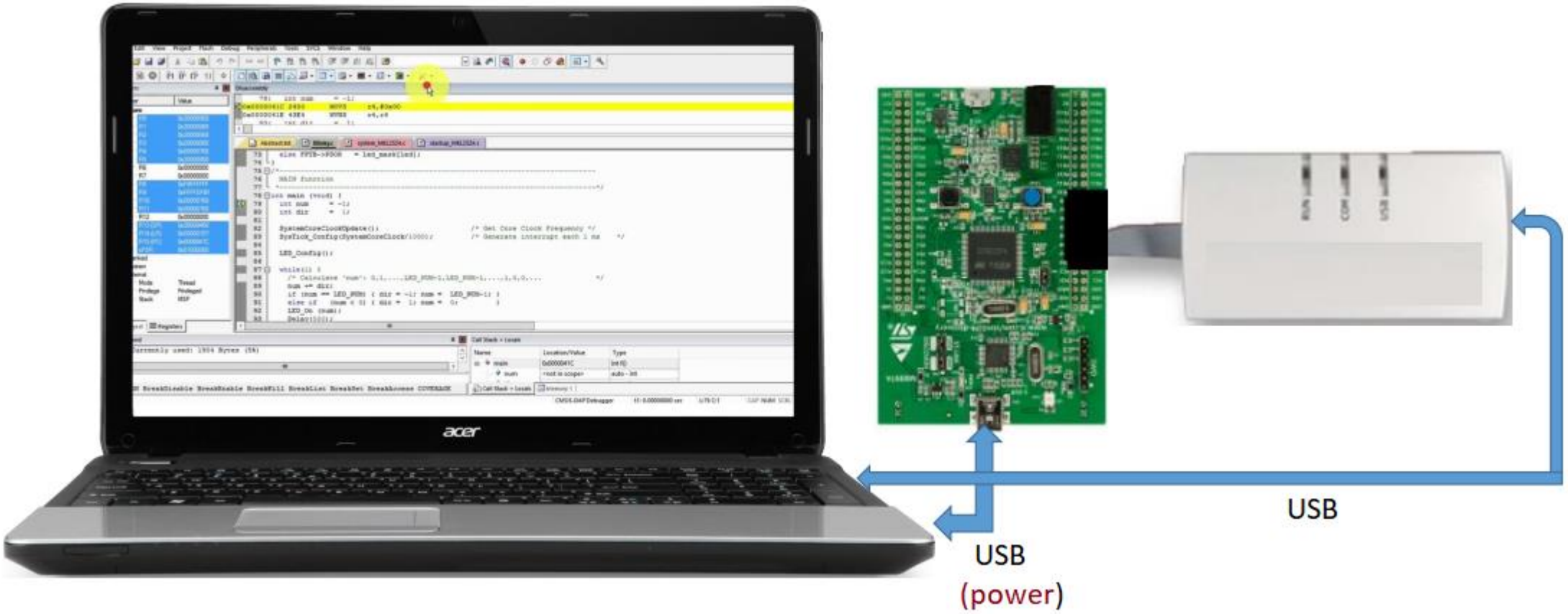
Position Controller



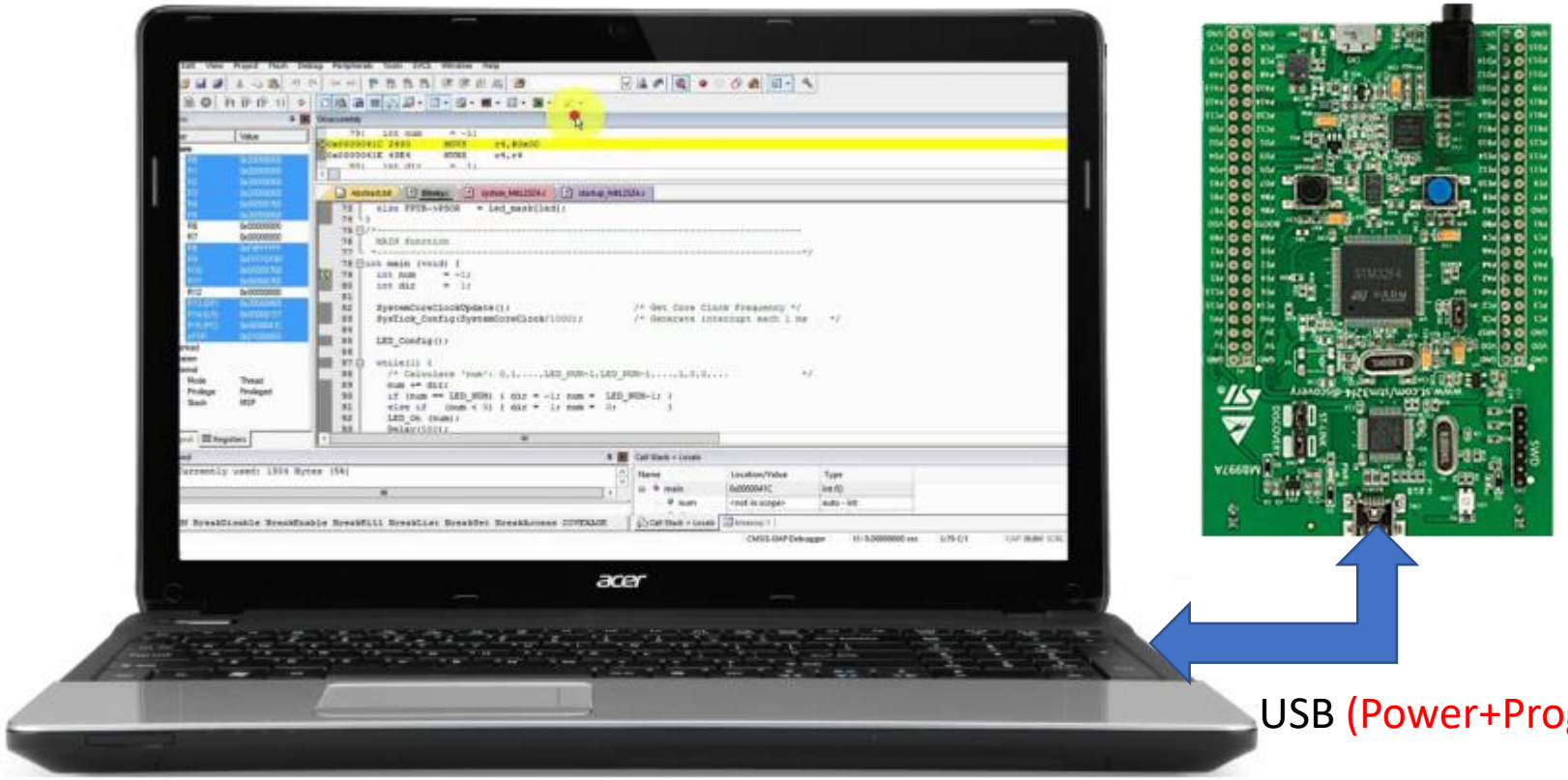
Manual Docking between two FloatSats.

Toolchain for Programming Discovery board(stm32f407G)

Programming Discovery board(stm32f407G)

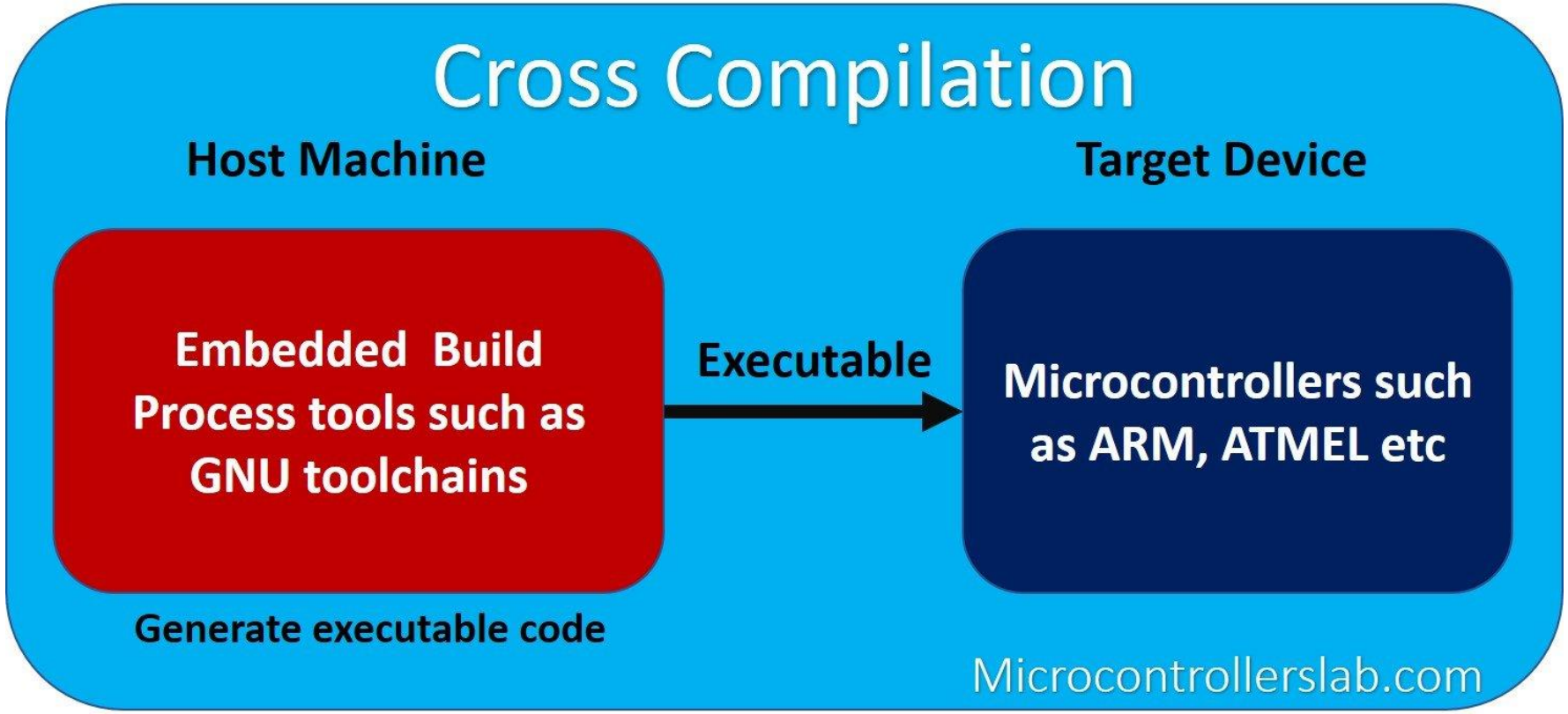


Programming Discovery board(stm32f407G)

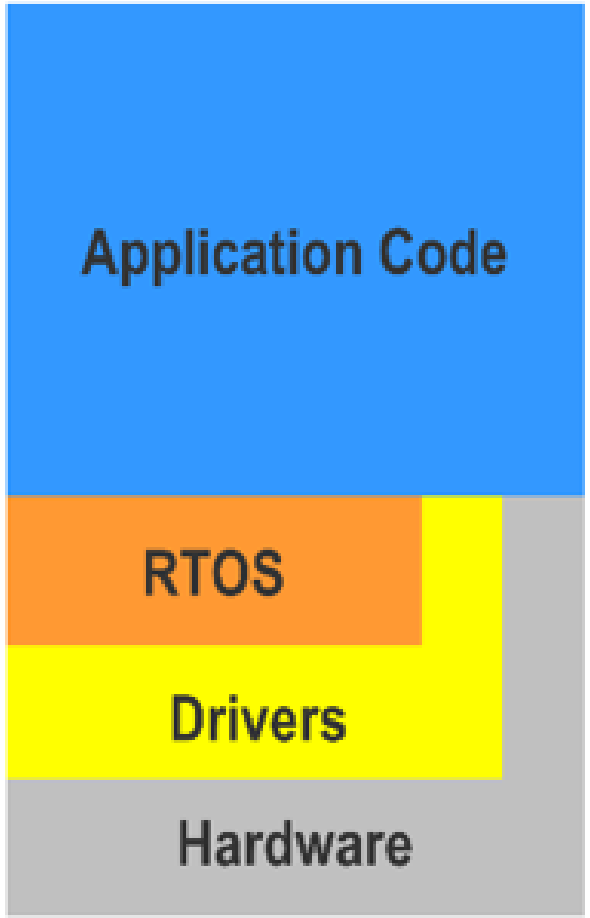


USB (Power+Programming)

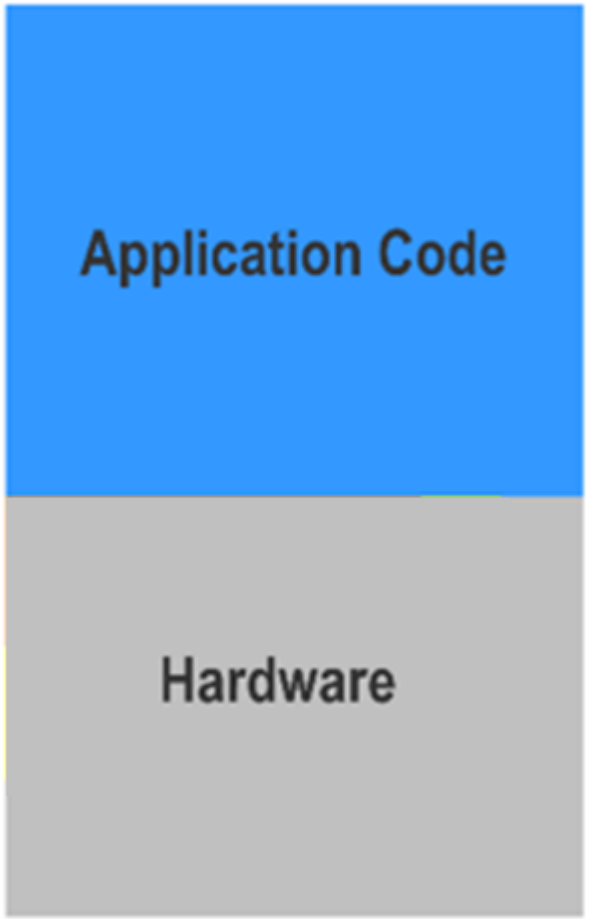
Embedded System Programming



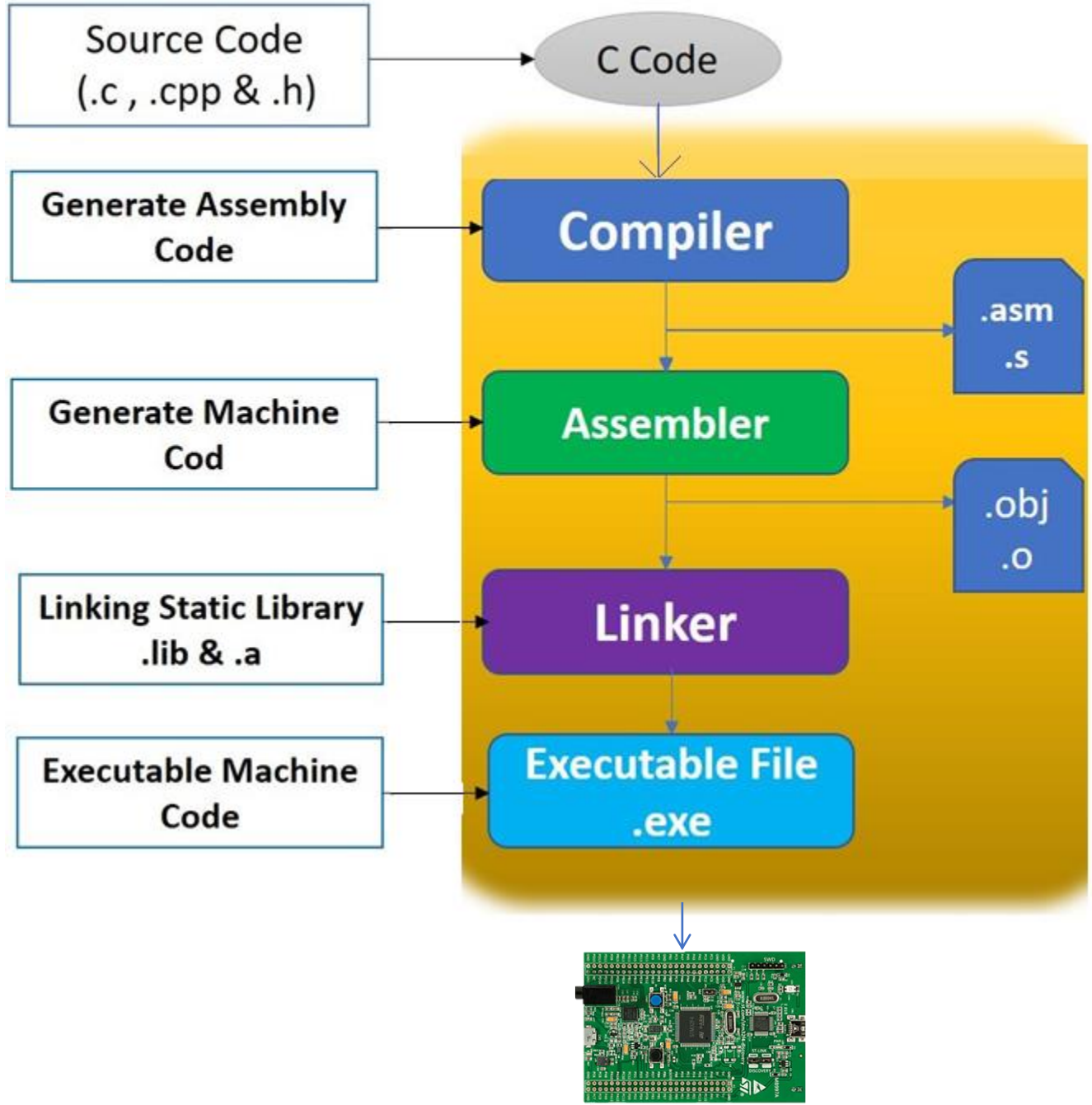
Embedded System with RTOS



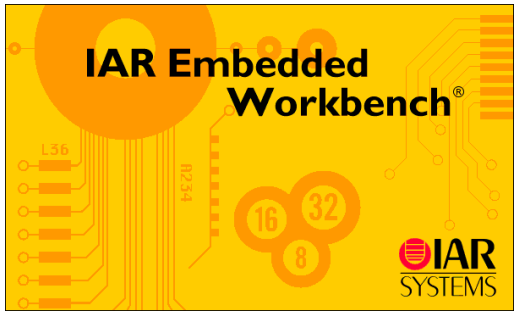
Bare-Metal Embedded System

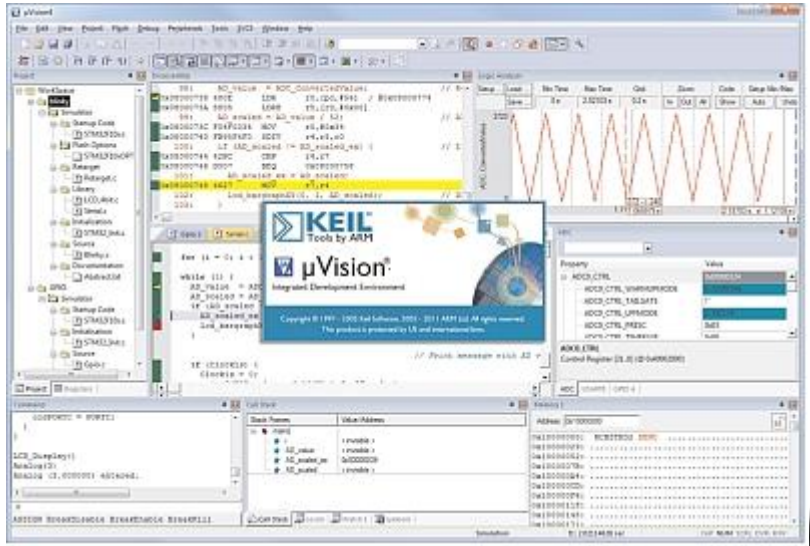


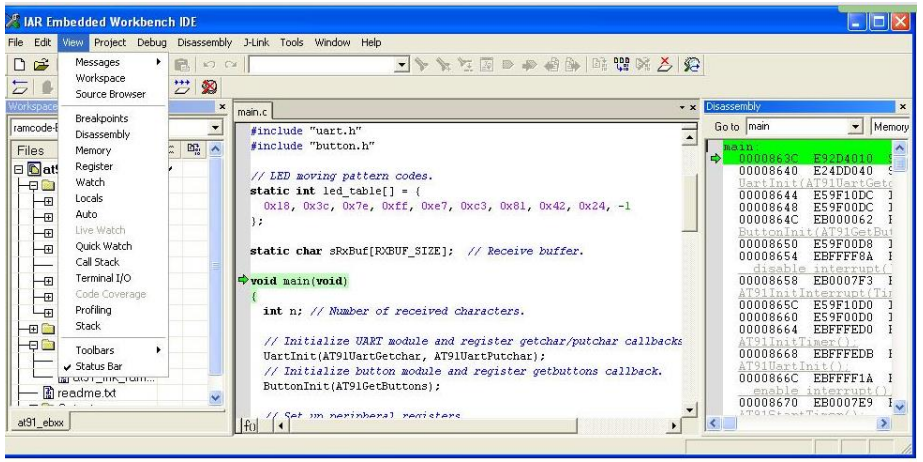
Toolchain for Programming Discovery board(stm32f407G)



ARM development Tools







open source processor emulator

What do we need to start programming the STM32F4?

- **IDE (Integrated Development Environment) for C/C++ Developers**: An IDE is a software application that provides comprehensive facilities to computer programmers for software development. An IDE normally consists of a **source code editor**, **build automation tools** and a **debugger**.

- **ARM GNU Toolchain**: An ARM GNU toolchain is a complete development environment for embedded C/C++ development on ARM. The toolchain normally consists of **GNU C and C++ compilers**, **GNU assembler & linker**, **C and C++ runtime libraries** and **GNU debugger**.

- **ST-LINK Utility** software for fast in-system programming (ISP) of the STM32 microcontroller via the on-board **ST-LINK/V2**.

- **GDB server for debugging.**

- **(Optional) Real Time Operating System Project Library configured to use the Firmware Library of the STM32F4 and a Template Project linked with the library.**

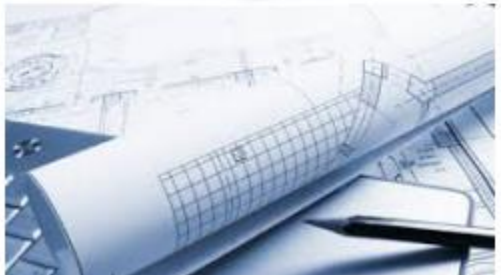
ARM

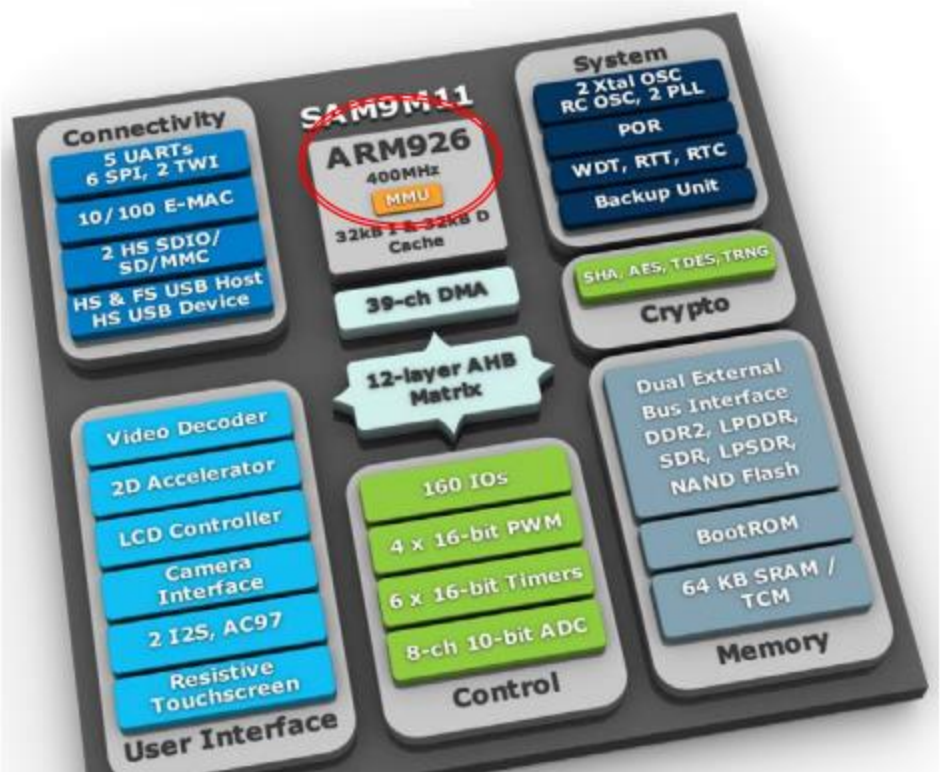
Fundamentals of the most popular embedded architecture in the World Today

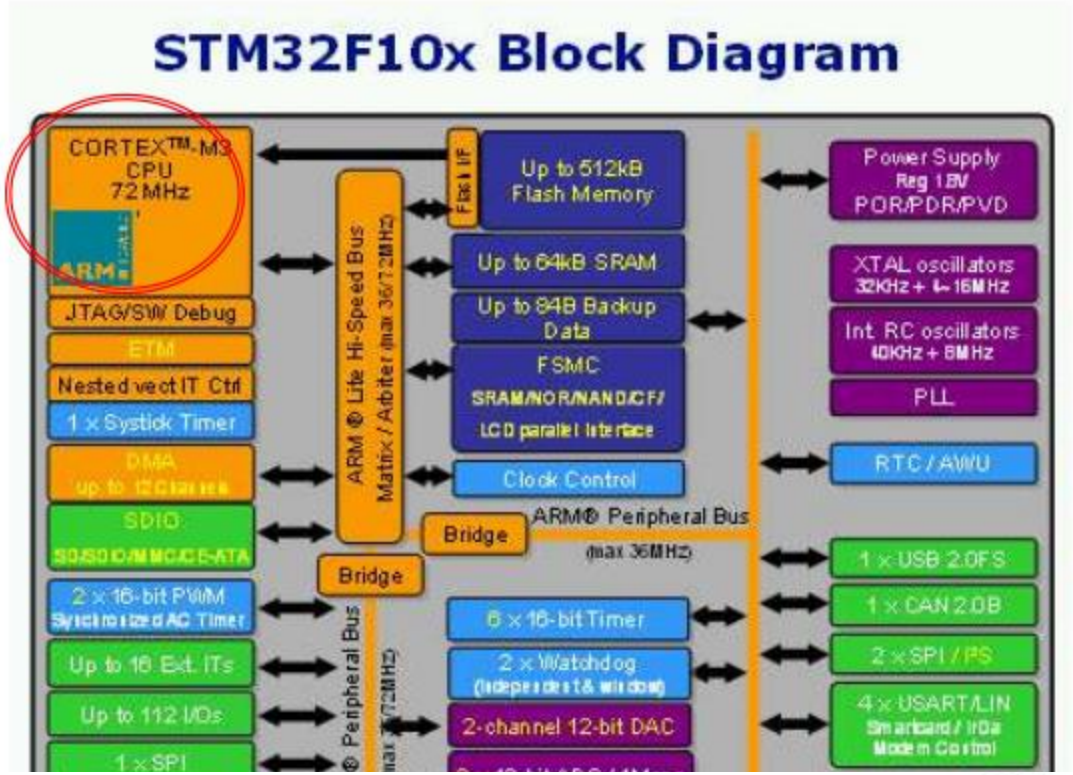
ARM

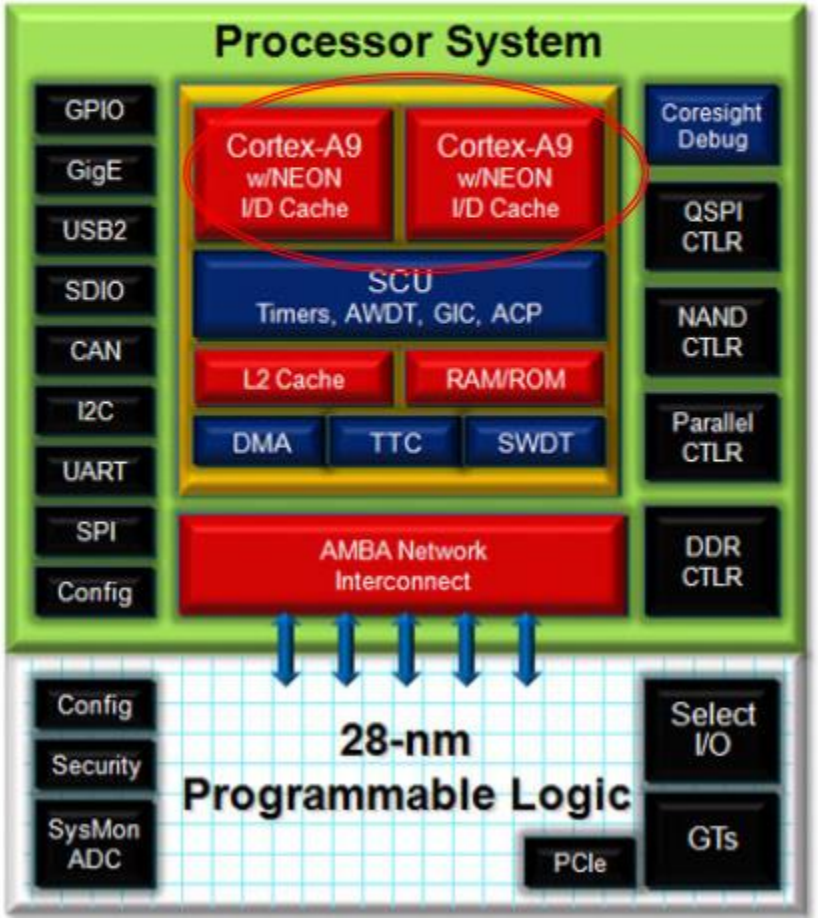
ARM designs microprocessor technology that lies at the heart of advanced digital products, from mobile phones and digital cameras to games consoles and automotive systems, and is leading intellectual property (IP) provider of high-performance, low-cost, power-efficient RISC processors, peripherals, and system-on-chip (SoC) designs ,

- ARMs business Model is quite different than the rest of Microelectronic Industry.
- ARM Ltd. does not manufacture or sell any physical chips. Instead, it provides other semiconductor companies with intellectual property (IP) solutions in the form of ARM architecture cores or systems on a chip (SOCs).
- The licensees of the ARM core can then optimize, customize, and manufacture the IP for their particular use cases or customer requirements



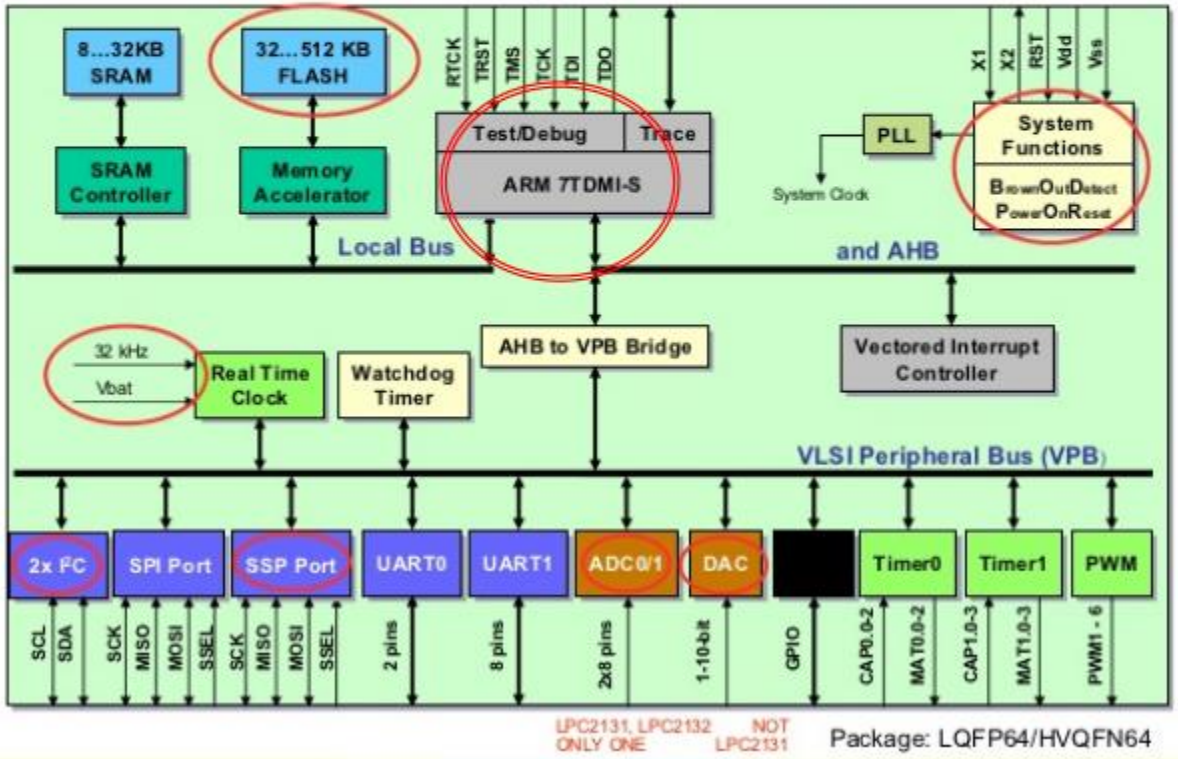

Connectivity
5 UARTs
6 SPI, 2 TWI
10/100 E-MAC
2 HS SDIO/
SD/MMC
HS & FS USB Host
HS USB Device
SAM9M11
ARM926
400MHz
MMU
Cache
39-ch DMA
System
2 Xtal OSC
RC OSC, 2 PLL
POR
WDT, RTT, RTC
Backup Unit
SHA, AES, TDES, TRNG
Crypto
12-layer AHB
Matrix
Video Decoder
2D Accelerator
LCD Controller
Camera
Interface
2 I2S, AC97
Resistive
Touchscreen
User Interface
160 IOs
4 x 16-bit PWM
6 x 16-bit Timers
8-ch 10-bit ADC
Control
Dual External
Bus Interface
DDR2, LPDDR,
SDR, LPSDR,
NAND Flash
BootROM
64 KB SRAM /
TCM
Memory


STM32F10x Block Diagram
CORTEX™-M3
CPU
72 MHz
JTAG/SW Debug
ETM
Nested vect IT Ctrl
1 x Systick Timer
DMA
SDIO
2 x 16-bit PWM
Up to 16 Ext. ITs
Up to 112 I/Os
1 x SPI
ARM® Lite Hi-Speed Bus
Matrix / Arbiter
Up to 512kB
Flash Memory
Up to 64kB SRAM
Up to 84B Backup
Data
FSMC
Clock Control
Bridge
ARM® Peripheral Bus
(max 36MHz)
6 x 16-bit Timer
2 x Watchdog
2-channel 12-bit DAC
Power Supply
Reg 1.8V
POR/PDR/PVD
XTAL oscillators
Int. RC oscillators
PLL
RTC / AWU
1 x USB 2.0FS
1 x CAN 2.0B
2 x SPI / I²S
4 x USART/LIN



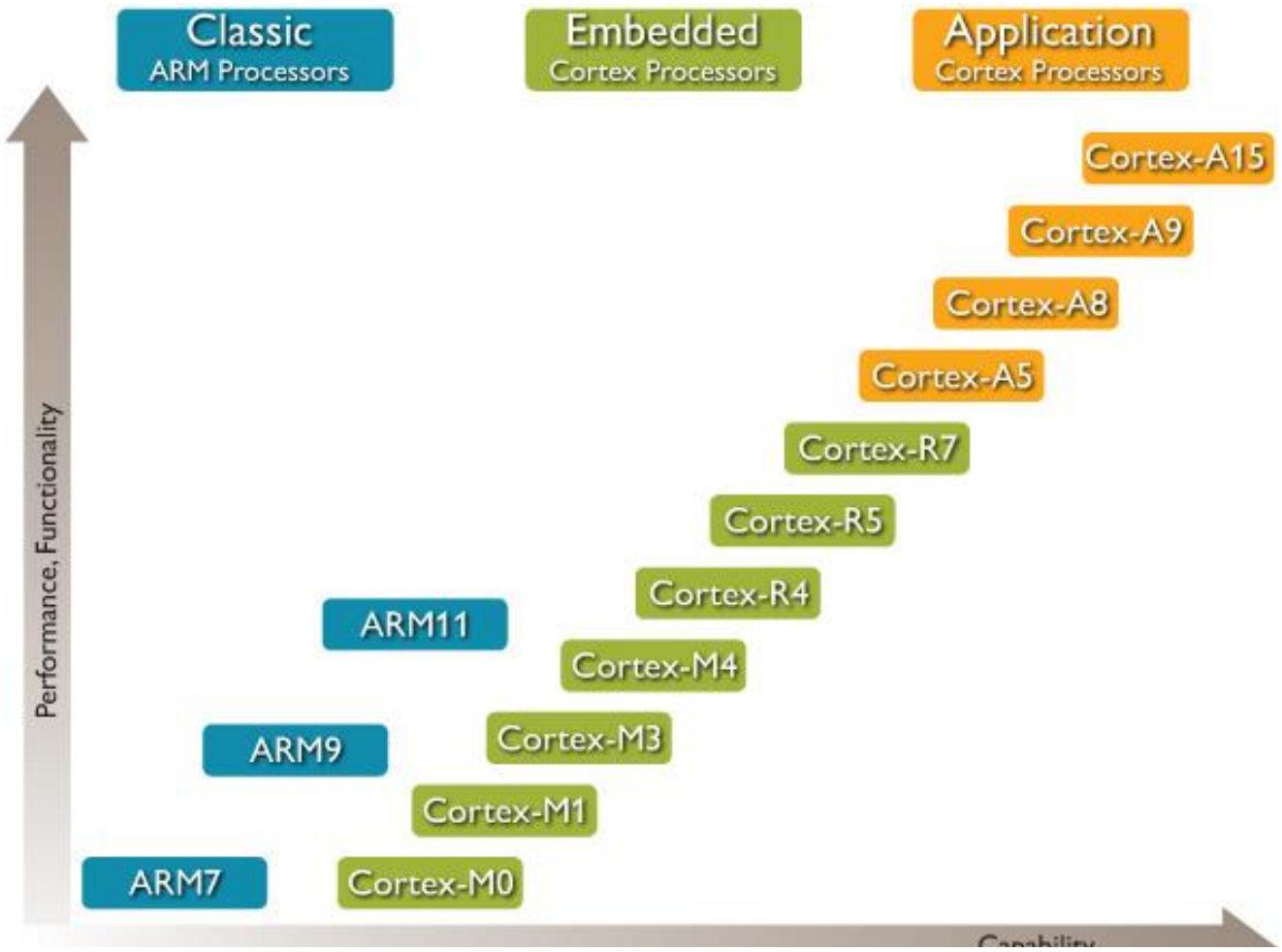
NXP Semiconductors

LPC213x Block Diagram



ARM Powered Products

ARM Processors



UM1472
User Manual

STM32F4DISCOVERY

STM32F4 high-performance discovery board

Table 5. MCU pin description versus board function (page 1 of 10)

MCU pin			Board function													
Main function	Alternate functions	LQFP100	CS43L22	MP45DT02	LIS302DL	Pushbutton	LED	SWD	USB	OSC	Free I/O	Power supply	CN5	CN2	P1	P2
BOOT0	VPP	94														21
NRST		14				RESET		NRST						5	6	
PA0-WKUP	USART2_CTS/ USART4_TX/ ETH_MII_CRS/ TIM2_CH1_ETR/ TIM5_CH1/ TIM8_ETR/ ADC123_IN0/ WKUP	23				USER									12	
PA1	USART2_RTS/ USART4_RX/ ETH_RMII_REF_CLK/ ETH_MII_RX_CLK/ TIM5_CH2/ TIMM2_CH2/ ADC123_IN1	24													11	
PA2	USART2_TX/ TIM5_CH3/ TIM9_CH1/ TIM2_CH3/ ETH_MDIO/ ADC123_IN2	25													14	

The STM32F4DISCOVERY Extension Board

Hardware for the Exercises

Exercise:

Programming Toolchain and Debugging Discovery Board (stm32f407)

Using GPIO in uController

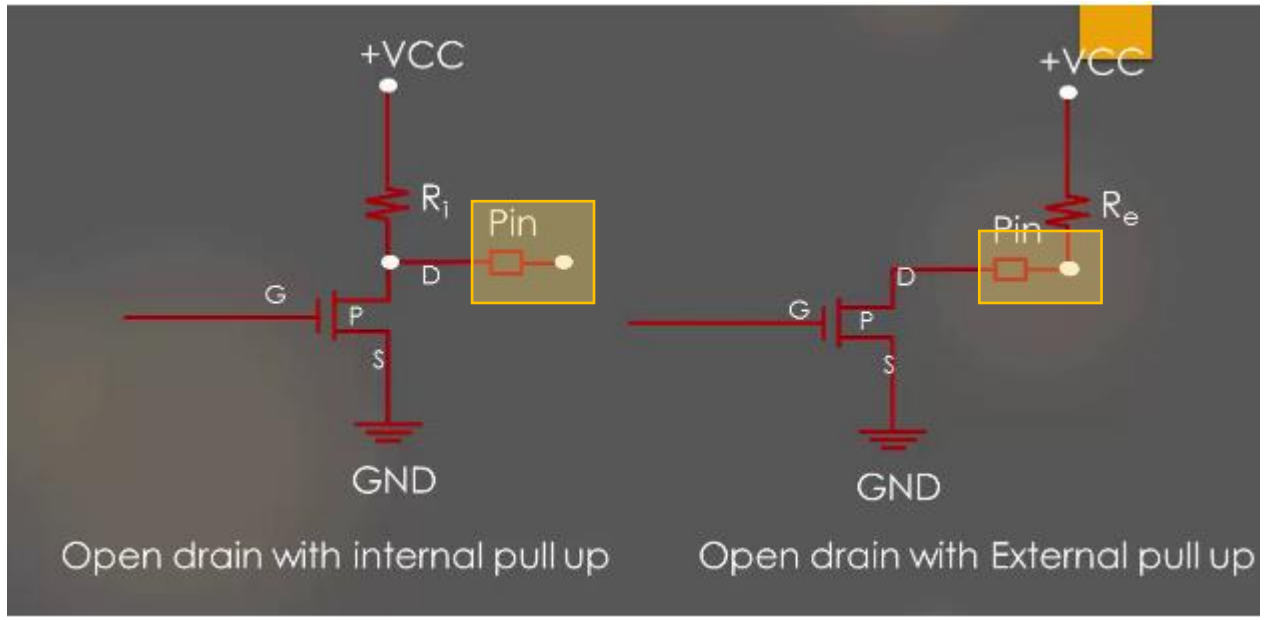
Digital Input:

1. Enable the GPIO port peripheral clock
2. Set input mode
3. Internal pull-up/-down resistors
4. Read pin value

Digital Output:

1. Enable the GPIO port peripheral clock
2. Set output mode
3. Configure output type
4. Set output speed
5. Set output value

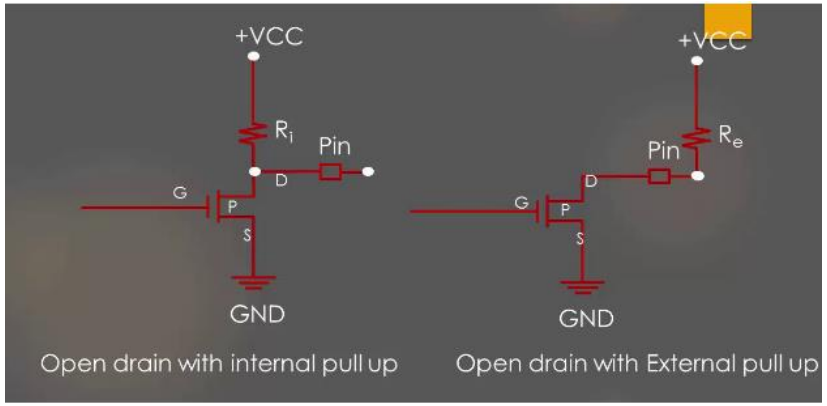
Using GPIO in uController



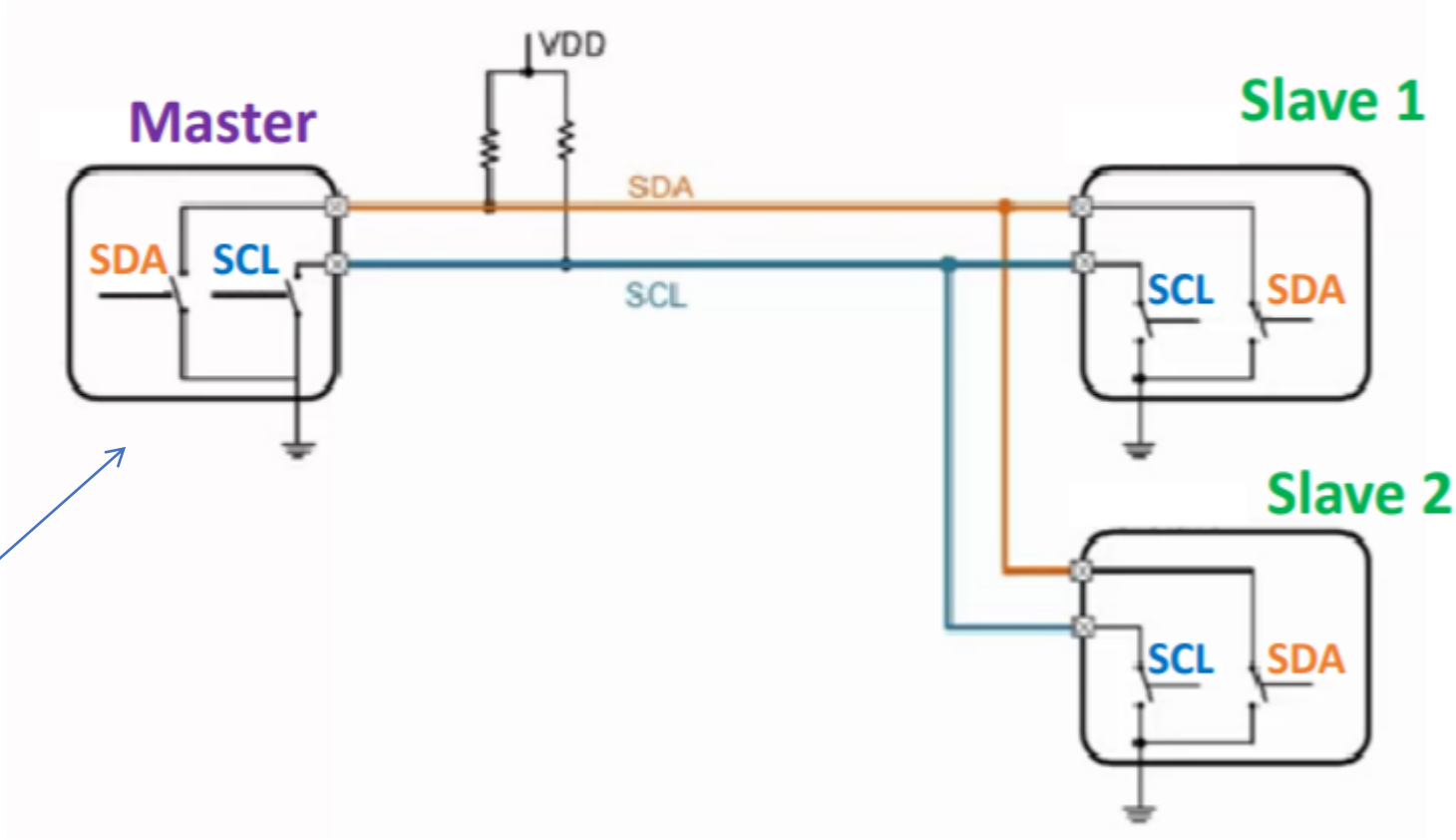
Reading: https://embeddedartistry.com/blog/2018/06/04/demystifying-microcontroller-gpio-settings/

Using GPIO in uController

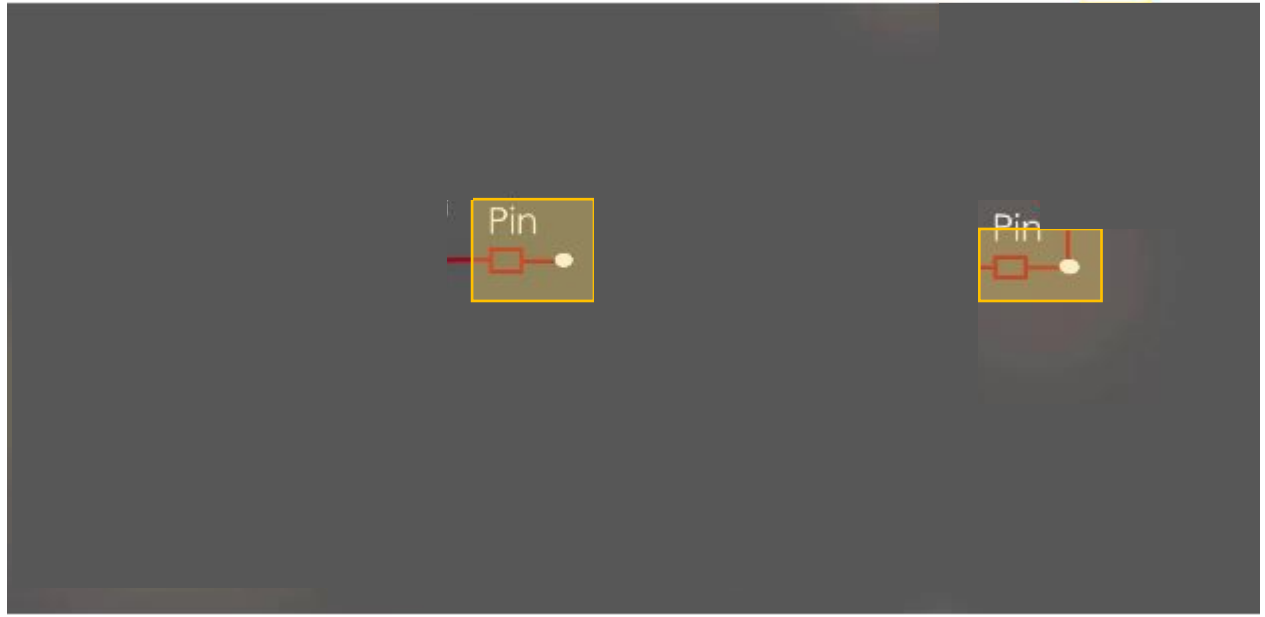
- Open-drain outputs are useful when multiple gates or pins are connected together, such as with the I2C bus.



I2c Protocol



Using GPIO in uController



Using GPIO in uController in a RTOS

Digital Input:

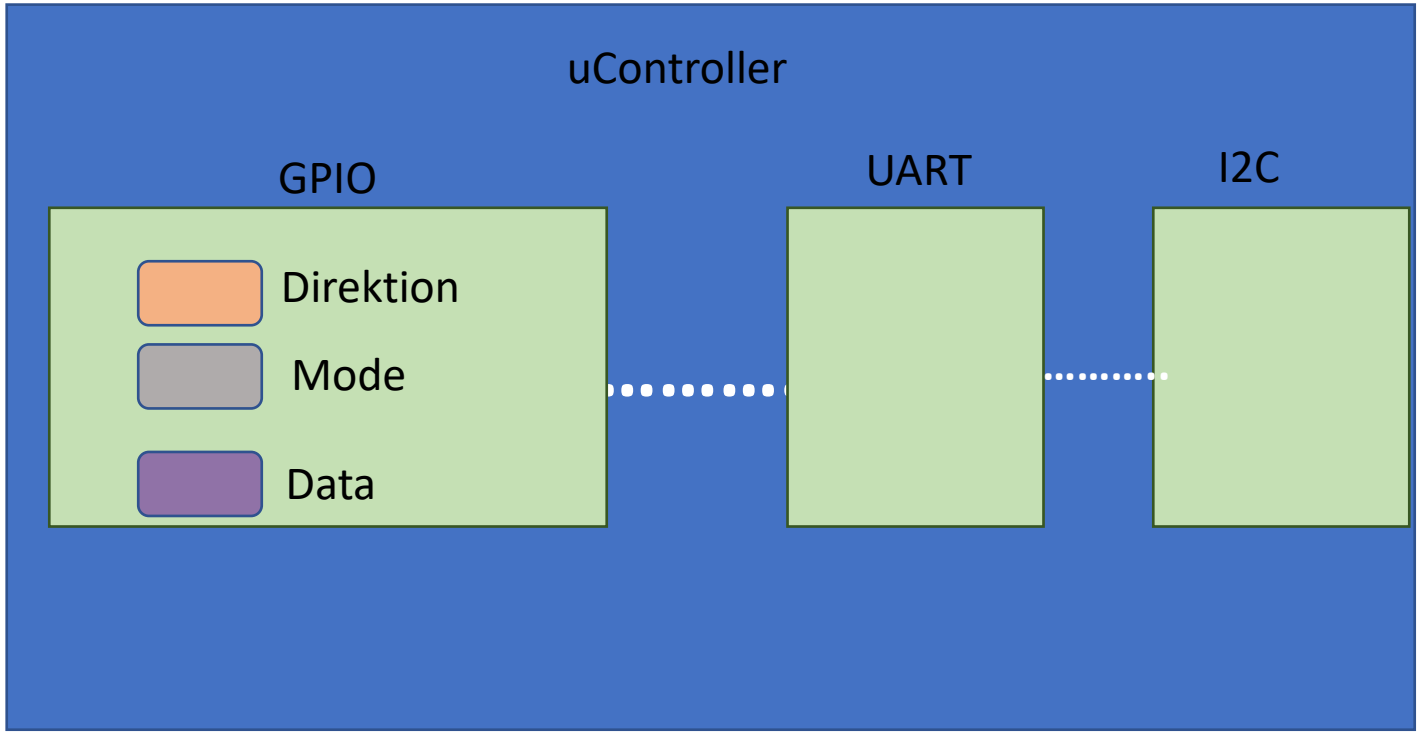
1. Enable the GPIO port peripheral clock
2. Set input mode
3. Internal pull-up/-down resistors
4. Read pin value

```
void Setup_LEDs(void)
{
  /* GPIOD Periph clock enable */
  RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOD, ENABLE);
  /* Configure PD12, PD13, PD14 and PD15 in output pushpull mode */
  GPIO_InitStructure.GPIO_Pin = GPIO_Pin_12 ;
  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
  GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
  GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
  GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
  GPIO_Init(GPIOD, &GPIO_InitStructure);

}
```

```
HAL_GPIO halgpio(GPIO_062);
halgpio.init(1,1,1);
```

Using GPIO in uController in a RTOS




```c
void Setup_LEDs(void)
{
  /* GPIOD Periph clock enable */
  RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOD, ENABLE);
  /* Configure PD12, PD13, PD14 and PD15 in output pushpull mode */
  GPIO_InitStructure.GPIO_Pin = GPIO_Pin_12 ;
  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
  GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
  GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
  GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
  GPIO_Init(GPIOD, &GPIO_InitStructure);

}
```

```c
HAL_GPIO halgpio(GPIO_062);
halgpio.init(1,1,1);
```

halgpio.**readPin**();
halgpio.**setPins**(1 or 0);

• Hardware Abstraction Layer (HAL)

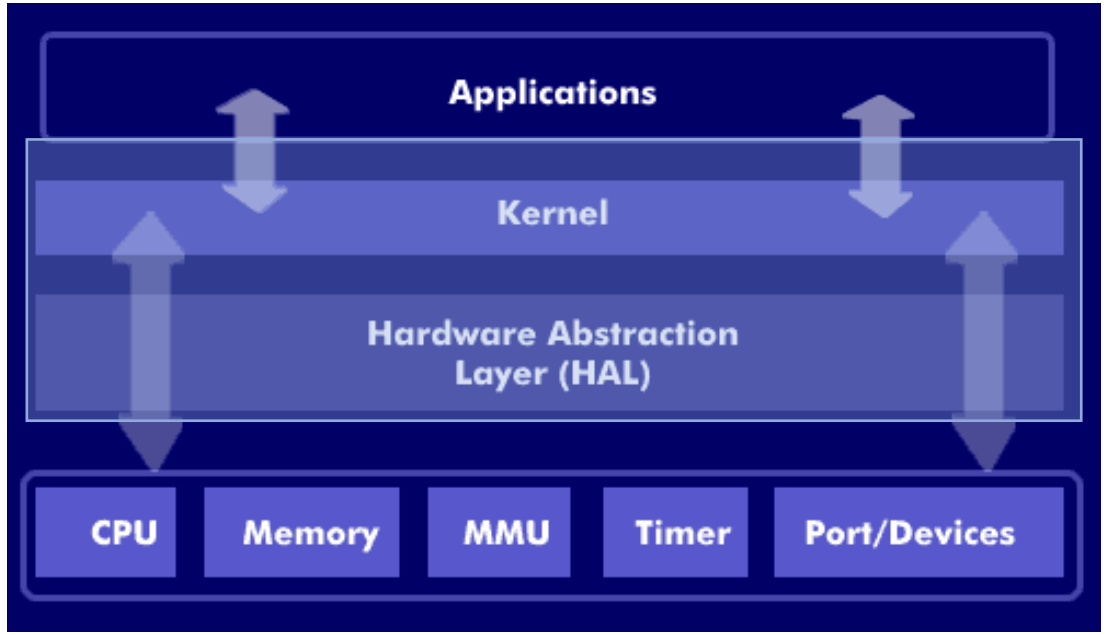
- A hardware abstraction layer (HAL) is a layer of programming that allows a computer OS to interact with a hardware device at a general or abstract level rather than at a detailed hardware level.

- Using HAL a developer can interact with the device in a more general way than it would otherwise.

What is HAL (hardware abstraction layer)

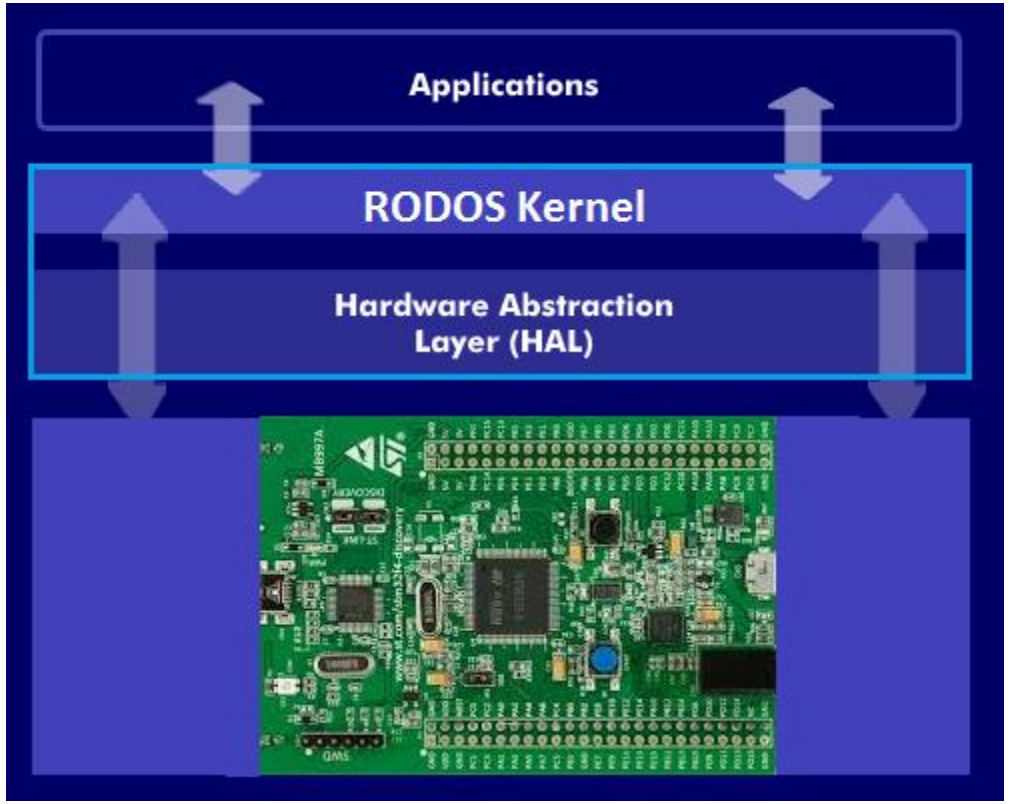
- The hardware abstraction layer (HAL) hides hardware differences from the operating system so that uniform code can be used for all hardware.

- All hardware looks the same to the operating system because it “sees” the hardware through the filtered glasses of the HAL.



Picture Source: https://networkencyclopedia.com/hardware-abstraction-layer-hal/

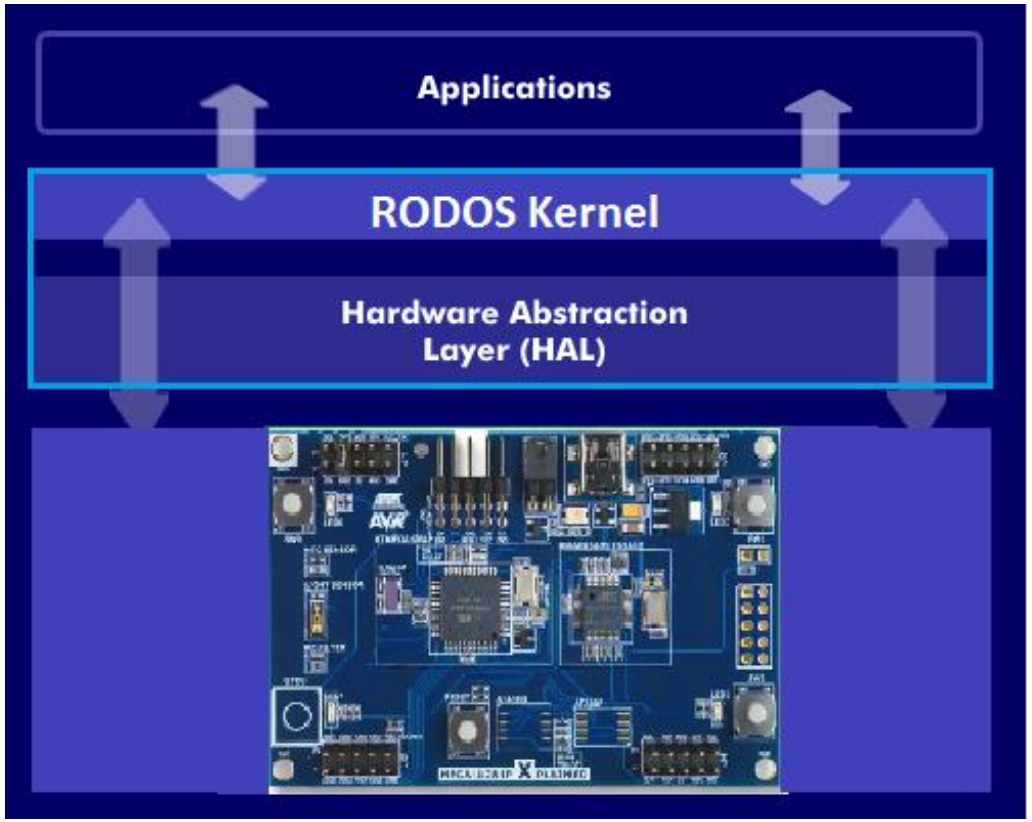
- Hardware Abstraction Layer (HAL)



HAL_GPIO(Pin Number)

HAL_UART(serialPortNo)

- Hardware Abstraction Layer (HAL)



HAL_GPIO(Pin Number)

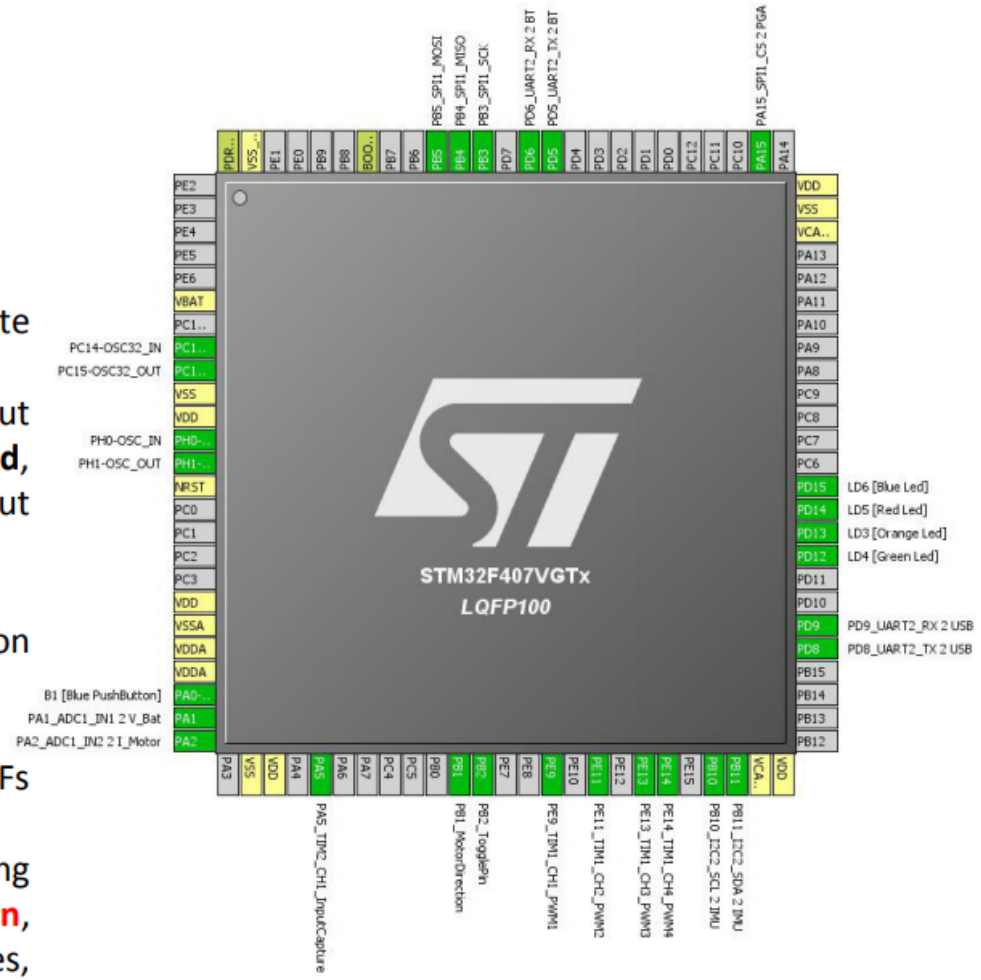
HAL_UART(serialPortNo)

General-Purpose I/O (GPIO):

- STM32F4DISCOVERY has **82x GPIO pins** on five available **ports A, B, C, D and E** plus additional **port H** with two GPIO pins only used for the HSE oscillator.
- Each **bit** of the **General-Purpose I/O (GPIO) ports** can be individually configured by software in several modes:
  - **Input mode.**
  - **Analog mode.**
  - **Output mode.**
  - **Alternate function mode.**
  - **External interrupt/event lines.**

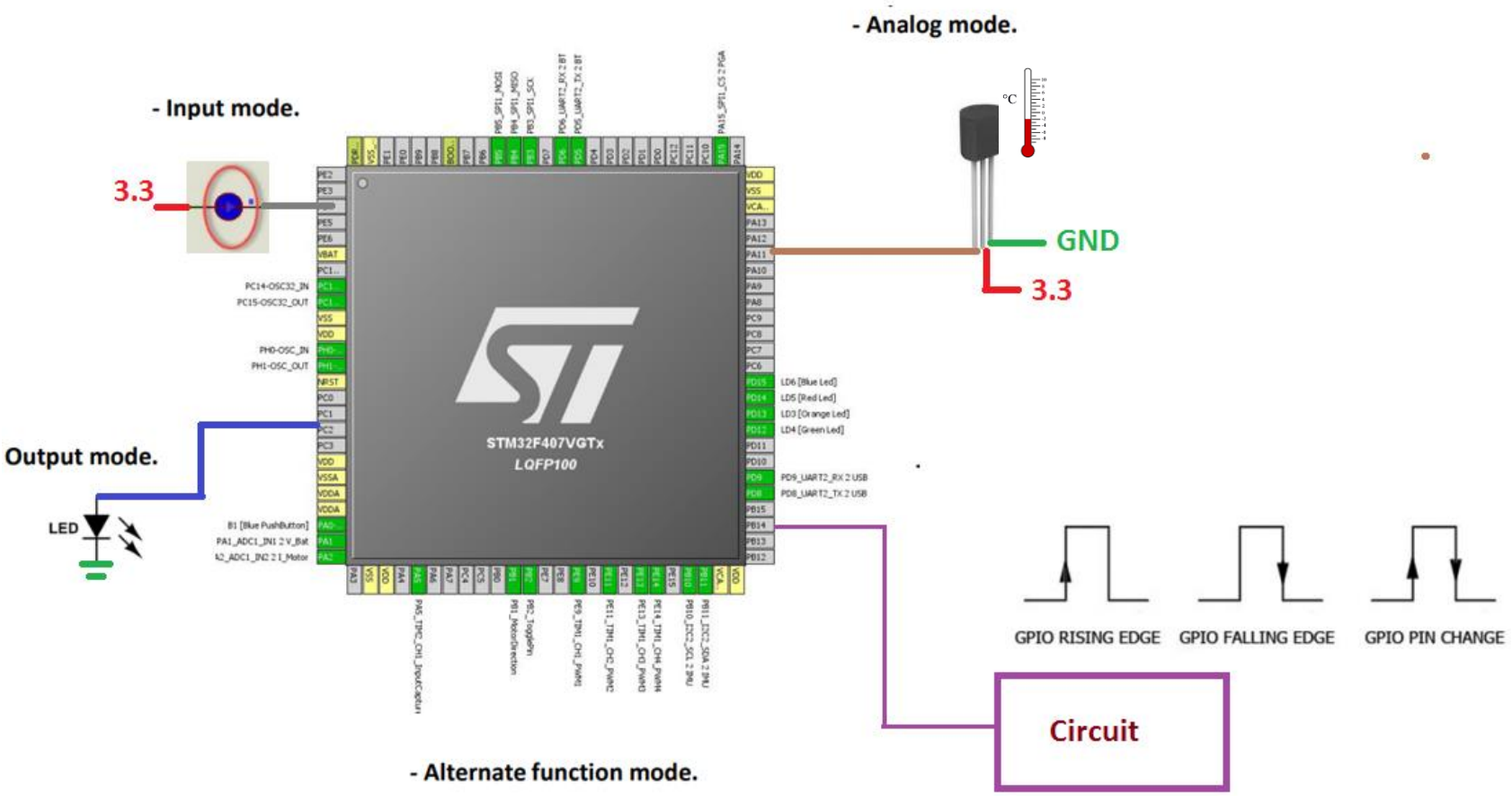
- **STM32F4 GPIO Main Features :**

- **Output states**: **push-pull** or **open drain + pull-up/down**.
- **Output data** from **output data register** or **peripheral** (alternate function output).
- **Speed** selection for each I/O GPIO **Output**. Maximum output frequency can be chosen as **2 MHz Low speed**, **25 MHz Medium speed**, **50 MHz Fast speed** or **100 MHz High speed** on 30 pF (80 MHz Output max speed on 15 pF).
- **Input states**: **floating**, **pull-up/down** or **analog.**
- **Input data** to **input data register** or **peripheral** (alternate function input).
- **Analog function.**
- **Alternate function** input/output selection registers (at most 16 AFs per I/O).
- All ports have **external interrupt/event capability** on the following events: **Rising edge trigger detection**, **Falling edge trigger detection**, **Rising/Falling edge trigger detection**. To use external interrupt lines, the port must be configured in **input mode**.



General-Purpose I/O (GPIO):

- STM32F4DISCOVERY has **82x GPIO pins** on five available **ports A, B, C, D and E** plus additional **port H** with two GPIO pins only used for the HSE oscillator.
- Each **bit** of the **General-Purpose I/O (GPIO) ports** can be individually configured by software in several modes:



- Alternate function mode.

PA2	USART2_TX/ TIM5_CH3/ TIM9_CH1/ TIM2_CH3/ ETH_MDIO/ ADC123_IN2

· External interrupt/event lines.

RODOS (Realtime Onboard Dependable Operating System):

- Preemptive Real Time Operating System
- Open Source Project
- Developers:
  - German Aerospace Center (DLR)
  - Department for Aerospace Information Technology at the University of Würzburg.
- C++ API
- Easy to schedule tasks
- Easy to share tasks
- Middleware with Topics and Subscribers
- Gateway brings Topics to other RODOS nodes

```
class Telemetry: public Thread {

public:

Telemetry(const char* name) : Thread(name) {}

void init() {GreenLED.init(true, 1, 0);}

void run() {
            while (1) {
                        GreenLED.setPins(~GreenLED.readPins());
                        PRINTF("Hello Rodos, the time now is %f \r\n",SECONDS_NOW());
                        suspendCallerUntil(NOW()+1000*MILLISECONDS);}
            }
};
Telemetry Telemetry("Telemetry");
```

Time NOW() in seconds as double

Returns the time now in the internal time resolution (eg. Nanoseconds).

Suspend the thread until a given time or a wake up by a call to the method resume.

RODOS HAL:

- **The HAL (Hardware Abstraction Layer) drivers layer provides a generic multi instance simple set of APIs (application programming interfaces) to interact with the upper layers (application, libraries and stacks).**

```
class GenericIOInterface {
private:
    IOEventReceiver* ioEventReceiver;
    Thread* callerReadFinished;
    Thread* callerWriteFinished;
    Thread* callerDataReady;

protected:
void* context;

public:
    GenericIOInterface(); // create all drivers static but do not init there

    /**** Implemented by each interface ****/
    virtual int init(...)   {return 0;}  ///< Virtual, but do not call using a pointer, each init differs
    virtual void reset() { }
    virtual int config(...) {return 0;}  ///< Virtual, but do not call using a pointer, each config differs
    virtual int status(...) {return 0;}  ///< Virtual, but do not call using a pointer, each status differs

    virtual bool isWriteFinished() = 0;
    virtual bool isReadFinished()   {return false;}
    virtual bool isDataReady()   {return false;}

    virtual int write(...)      {return 0;}
    virtual int read(...)       {return 0;}
    virtual int writeRead(...)  {return 0;}

    void suspendUntilReadFinished(int64_t reactivationTime = END_OF_TIME);
    void suspendUntilWriteFinished(int64_t reactivationTime = END_OF_TIME);
    void suspendUntilDataReady(int64_t reactivationTime = END_OF_TIME);

    /******* may be overridden by the user or received through IOEventREceiver, called by corrsponding driver  *****/
    virtual void upCallWriteFinished(); // last transmit order is concluded
    virtual void upCallReadFinished();  // last receive order is concluded
    virtual void upCallDataReady();     // data has arrived

    void setIoEventReceiver(IOEventReceiver* receiver);
};
```

RODOS HAL (GPIO):

```
class HW_HAL_GPIO; /* hal_gpio.h */

class HAL_GPIO : public GenericIOInterface {
private:
HW_HAL_GPIO *context;
public:
HAL_GPIO(GPIO_PIN pinIdx = GPIO_000);

int32_t init(bool isOutput = false, uint32_t numOfPins = 1, uint32_t initVal = 0x00);
void reset();
int32_t config(GPIO_CFG_TYPE type, uint32_t paramVal);

bool isDataReady();
bool isWriteFinished() {return true;}
bool isReadFinished() {return true;}

void setPins(uint32_t);
uint32_t readPins();

/* interrupts supported by: STM32F4 */
//When rising and/or falling edge occurs isDataReady() == true
void interruptEnable(bool enable);  //After this function, isDataReady() returns false, until the configured edge
occurs on the gpio pin. For e.g if you want to suspend a thread until a rising edge event is detected. Configure
the pin for rising edge and enable the interrupt for the corresponding pin. Then you can suspend the thread using
suspendUntilDataReady() function.
void resetInterruptEventStatus();
};
```

```
typedef enum {
GPIO_CFG_OUTPUT_ENABLE = 0,
GPIO_CFG_NUM_OF_PINS,
GPIO_CFG_PULLUP_ENABLE,
GPIO_CFG_PULLDOWN_ENABLE,
GPIO_CFG_IRQ_SENSITIVITY
} GPIO_CFG_TYPE;
```

```
typedef enum {
GPIO_IRQ_SENS_BOTH = 0,
GPIO_IRQ_SENS_RISING,
GPIO_IRQ_SENS_FALLING
} GPIO_IRQ_SENSITIVITY;
```

Example:

```
HAL_GPIO GreenLED(GPIO_060); /* declare HAL_GPIO for GPIO_060 = PD14 (Green LED), please refer to hal_gpio.h for correct pin mapping */
GreenLED.init(true, 1, 0);   /* initialization of the HAL object should be called one time only in the project*/
GreenLED.setPins(~GreenLED.readPins());          /* toggle the GPIO pin PD14 */
```